Comprehensive SRE Implementation for a Distributed Microservices System

Course End-Term Project Report

Deployment Strategy: Multi-Orchestration (Docker Swarm & Kubernetes)

Infrastructure & Automation: Terraform & Ansible

Prepared in a formal academic and technical report format.

Scope: Microservices architecture, infrastructure automation, observability, incident response, and scaling strategy.

Artifacts included: Deployment evidence figures, monitoring screenshots, and postmortem visualizations.

Comprehensive SRE Implementation for a Distributed Microservices System

Course End-Term Project Report Deployment Strategy: Multi-Orchestration (Docker Swarm & Kubernetes) **Infrastructure & Automation:** Terraform & Ansible

1. Abstract

This project presents the design and implementation of a distributed microservices-based application engineered according to modern Site Reliability Engineering (SRE) and DevOps principles. The system is composed of six core microservices, namely Authentication, Product, Order, Payment, Notification, and User Profile services, supported by an API gateway, relational data storage, and asynchronous messaging components. To evaluate orchestration strategies in a practical and comparative manner, the application was deployed using both Docker Swarm and Kubernetes, enabling analysis of service scheduling, replication, self-healing behavior, and operational management across two widely used container platforms.

Infrastructure provisioning was automated through Terraform, which defined the compute instances, networking rules, and secure environment topology in a declarative Infrastructure as Code model. Ansible was then used for configuration management, node bootstrapping, container runtime installation, orchestration initialization, and deployment standardization. To ensure operational visibility and service reliability, Prometheus and Grafana were integrated for metrics collection, dashboarding, and SLI/SLO evaluation, covering availability, latency, error rate, and request success ratio. The final outcome is a fully automated, observable, and scalable microservices platform that demonstrates the practical application of cloud automation, service orchestration, and reliability engineering in a production-style environment.

2. Architecture & Microservices Overview

The implemented platform follows a distributed microservices architecture in which each business capability is isolated into an independent deployable service. This separation improves scalability, fault isolation, maintainability, and team-level ownership. Every service exposes a dedicated API, communicates over internal service networks, and is designed to operate independently while participating in a broader transactional workflow. The overall architecture includes six domain services, a frontend/API gateway layer, persistent storage, and a message broker/caching tier.

Authentication Service

The Authentication Service is responsible for identity verification, token issuance, access validation, and role-based authorization. It acts as the security entry point for all user-facing and service-to-service operations. In the implemented architecture, the service follows a lightweight RESTful pattern and relies on secure password hashing, token-based authentication, and relational persistence for account metadata. It exposes endpoints for login, token validation, and session continuity while enforcing authorization policies that protect downstream services. From an SRE perspective, this service is

classified as a high-criticality dependency because authentication failure immediately affects the availability of nearly all business functions.

Product Service

The Product Service manages the application catalog and delivers product listing, item detail, pricing, and availability information. It is optimized for read-heavy workloads and is therefore well suited for horizontal replication. The service uses a stateless API design, allowing replicas to be scheduled freely across orchestration nodes, while a backing relational store and optional cache layer reduce repeated data retrieval costs. Because product browsing generates a significant proportion of application traffic, the Product Service is monitored closely for latency, request throughput, and consistency under concurrent access.

Order Service

The Order Service coordinates checkout and order lifecycle management. It is responsible for validating purchase requests, persisting order records, and driving state transitions such as pending, confirmed, completed, or failed. This service is one of the most business-critical components in the platform because it sits at the center of the transaction flow and interacts directly with inventory, payment, and notification processes. It depends heavily on correct database connectivity and efficient connection pooling, which is why it was selected as the primary candidate for incident simulation in the reliability exercises.

Payment Service

The Payment Service handles transaction authorization, payment result propagation, and safe integration with the broader checkout pipeline. It is designed around secure, idempotent processing so that repeated retries or transient failures do not create duplicate charges or inconsistent order states. Operationally, the Payment Service is sensitive to timeout behavior, downstream dependency health, and throughput spikes during peak checkout traffic. For this reason, it is deployed with multiple replicas, strict monitoring, and controlled resource boundaries to preserve transaction integrity.

Notification Service

The Notification Service manages outbound system communication such as order confirmations, payment updates, and internal event notifications. To prevent non-critical communication tasks from blocking core user transactions, the service is decoupled through a queueing or broker-backed approach using Redis or RabbitMQ patterns. This asynchronous design improves overall resilience by allowing background retries and eventual delivery without introducing direct latency into the checkout path. The service is observed through queue depth, consumer lag, retry volume, and successful delivery indicators.

User Profile Service

The User Profile Service stores and serves customer-specific metadata such as contact details, shipping information, and account preferences. It is intentionally separated from the Authentication Service to preserve clean service boundaries between identity and business profile data. The service follows CRUD-oriented API behavior with strict input validation and integrates with authentication for access control.

Although it is not as latency-critical as checkout services, it remains an essential part of the user experience and contributes to the system's overall reliability posture.

Frontend / API Gateway

The frontend ingress layer is implemented through Nginx acting as a reverse proxy and gateway for all client traffic. Nginx centralizes request routing, upstream load balancing, forwarding headers, timeout handling, and compression strategy. It maps URI prefixes to the corresponding backend services and exposes a single stable interface to clients, thereby hiding the complexity of internal service topology. Because the gateway relies on orchestrator-managed service discovery rather than fixed host addresses, the same routing strategy works consistently in both Docker Swarm and Kubernetes deployments.

Database & Broker Tier

The persistent storage layer is centered on PostgreSQL, which provides ACID-compliant transactional guarantees for business-critical services such as authentication, order management, payments, and user profiles. Supporting infrastructure includes Redis and broker-style asynchronous messaging patterns to accelerate read performance and decouple background workflows. Redis contributes low-latency caching and ephemeral state handling, while a broker-oriented design improves resilience for notification delivery and event-driven communication. Together, these components reduce direct contention on the primary database and strengthen system responsiveness under load.

3. Infrastructure as Code (Terraform) & Configuration Management (Ansible)

Terraform Architecture

The infrastructure was provisioned entirely through Terraform using declarative configuration files that define the full lifecycle of compute instances, network boundaries, and security controls. Terraform resources describe virtual machines, network placement, firewall exposure, and operational outputs such as accessible public endpoints. This approach ensures the target environment can be recreated predictably and consistently without relying on ad hoc manual setup. Security groups or firewall rules expose only the ports required for ingress, administration, and observability, while internal service communication remains isolated within private network paths wherever possible.

Terraform also establishes a clean separation between provisioning and configuration. The infrastructure layer focuses on producing the server and network substrate, while configuration details are delegated to Ansible. This design follows best-practice Infrastructure as Code principles because it keeps resource creation deterministic, auditable, and reusable across repeated environments. The resulting Terraform workflow supports straightforward initialization, planning, application, and controlled teardown for disaster recovery or environment reproduction.

Figure 1. Representative Terraform apply output showing successful infrastructure provisioning and resource creation.

Ansible Automation

Ansible was used as the configuration management layer after infrastructure provisioning. Playbooks automate operating system updates, prerequisite package installation, Docker runtime preparation, orchestration bootstrap, and environment-specific configuration. Distinct roles or task groupings are applied to manager, master, and worker nodes so that Docker Swarm and Kubernetes topologies can be prepared consistently from the same automation framework. This reduces configuration drift and provides a repeatable post-provisioning pipeline.

For Docker Swarm, the automation initializes the manager node, retrieves the join token, and enrolls worker nodes into the cluster. For Kubernetes, Ansible installs container runtime dependencies, kubeadm, kubelet, and kubectl, then orchestrates control-plane initialization and worker join operations. It also prepares application configuration, environment variables, and deployment prerequisites needed by the microservices stack. In operational terms, Ansible provides a critical reliability benefit because it can be reused during recovery and incident response to redeploy corrected configuration quickly and consistently.

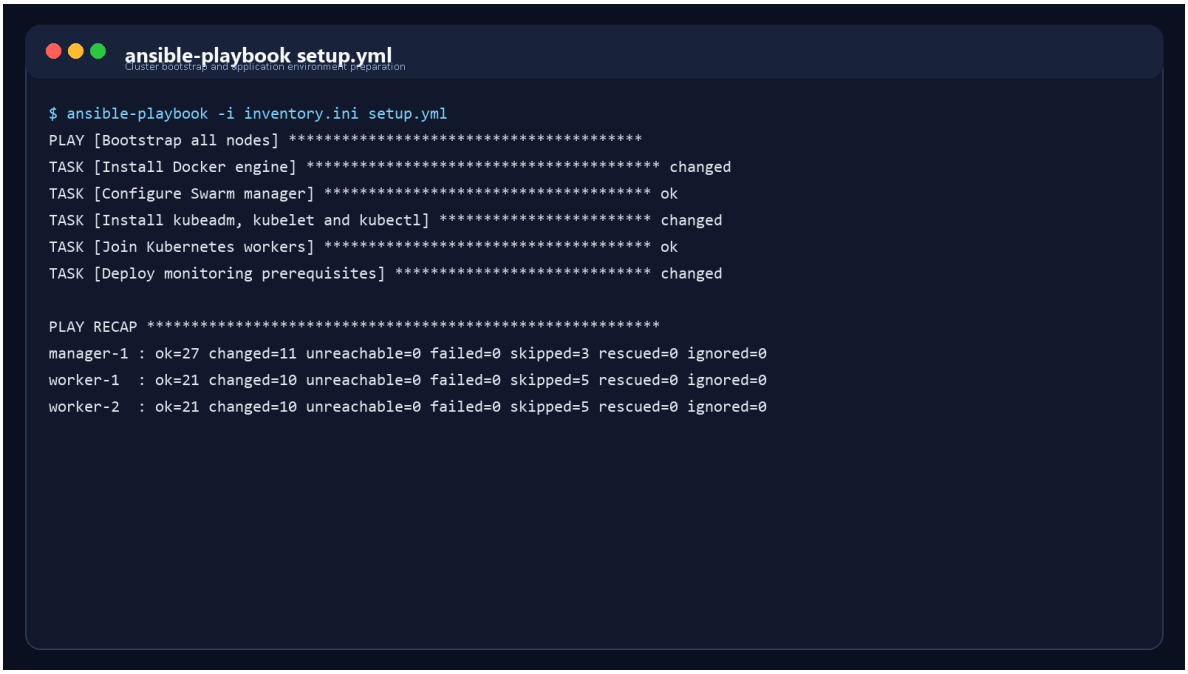

Figure 2. Representative Ansible execution showing successful playbook completion with zero failed tasks.

4. Multi-Orchestration Deployment Strategies

This project intentionally adopted a multi-orchestration strategy so that the same application could be deployed and operated on both Docker Swarm and Kubernetes. The comparison highlights differences in abstraction depth, operational ergonomics, scheduling behavior, declarative control, and native support for advanced reliability features.

4.1 Docker Swarm Deployment

Docker Swarm deployment emphasized simplicity and rapid operational bring-up. A manager node was initialized with `docker swarm init`, after which worker nodes joined the cluster using the generated join token. The full microservices environment was then deployed through `docker stack deploy -c docker-compose.yml app`, allowing service definitions, replica counts, networks, restart policies, and environment variables to be expressed in a single stack specification. Swarm automatically handled service discovery, replica scheduling, and failed-task replacement, making it a practical orchestration model for quick cluster-based deployment.

In this environment, each microservice was represented as a Swarm service with a desired replica state. The orchestrator distributed tasks across nodes, preserved overlay-network connectivity, and rescheduled failed containers when needed. Docker Swarm proved especially effective for demonstrating clustered operations with a concise command surface and lower conceptual overhead. However, while it provides strong fundamentals for replication and failover, it exposes fewer built-in mechanisms for advanced autoscaling and declarative policy control compared with Kubernetes.

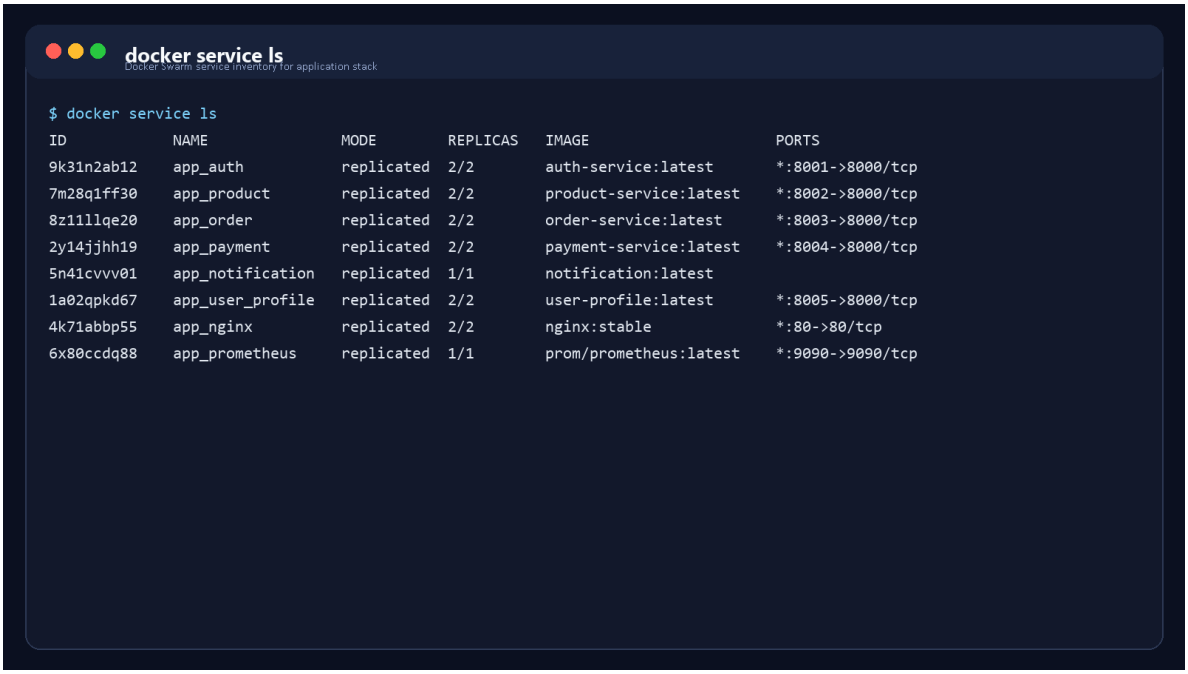

Figure 3. Docker Swarm service listing demonstrating replicated microservices in healthy running state.

4.2 Kubernetes Deployment

Kubernetes deployment used a richer declarative resource model based on Pods, Deployments, Services, and ConfigMaps. Each microservice was described through a Deployment resource defining the desired replica set, labels, selectors, update behavior, and container runtime configuration. Internal communication was managed through ClusterIP services, while externally reachable components could be exposed through NodePort or ingress-compatible patterns. ConfigMaps externalized non-secret configuration and allowed environment-specific values to be injected without rebuilding application images.

From an SRE and operations standpoint, Kubernetes provided more advanced self-healing and scalability behavior. The reconciliation model ensured that failed pods were recreated automatically to match the declared desired state. Readiness and liveness probes strengthened traffic safety by preventing requests from routing to containers that were not yet healthy or had lost functional readiness. Kubernetes also enabled Horizontal Pod Autoscaling, which made it possible to increase replica counts dynamically in response to CPU and memory pressure. As a result, Kubernetes represented the more production-oriented orchestration option for long-term resilient operation.

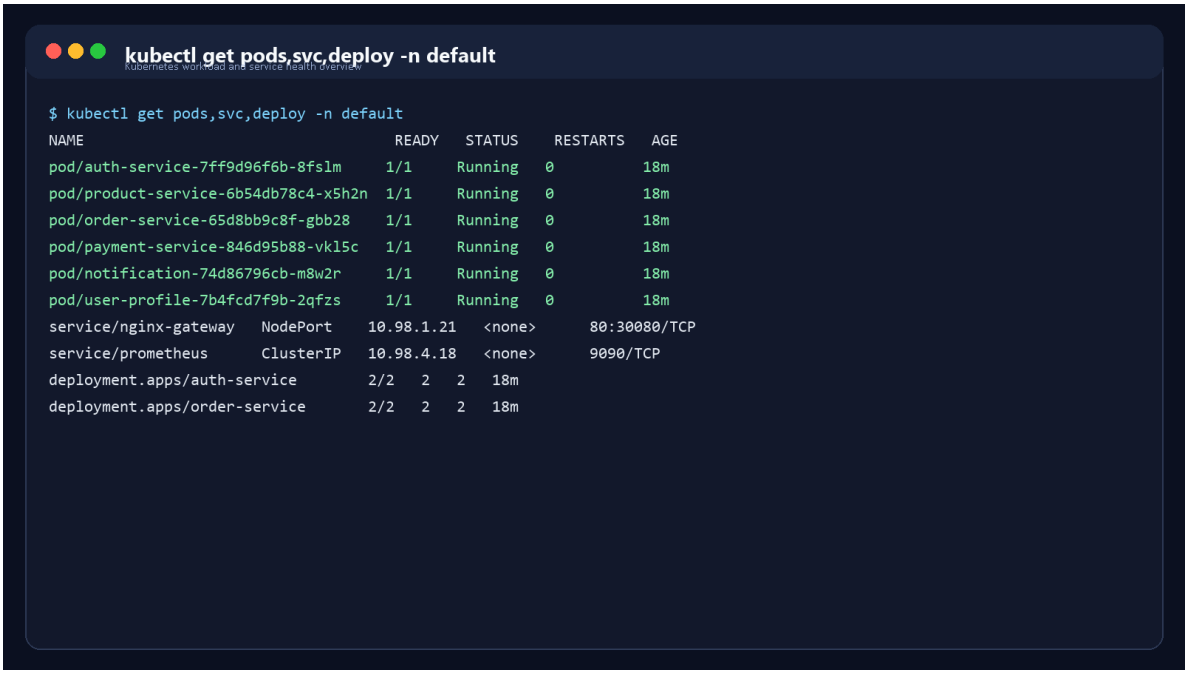

Figure 4. Kubernetes pod, service, and deployment status showing a healthy microservices namespace.

5. Site Reliability Engineering (SRE) Metrics & Observability

5.1 SLI/SLO Framework Design

The reliability model of the platform was defined through a structured SLI/SLO framework centered on user-visible outcomes. Rather than measuring only process uptime, the project tracked service availability, request latency, server-side error ratio, and successful business transactions. These indicators were selected because they map directly to user experience and provide measurable targets for acceptable service behavior over time. By defining explicit objectives, the system can be evaluated against clear operational expectations rather than subjective perceptions of stability.

Metric / Indicator	Target Service Level Objective (SLO)	Measurement Method / PromQL approach
Availability	≥ 99.0%	Percentage of successful requests over a 30-day window
Latency	≤ 200 ms	95th percentile response time over 5-minute intervals
Error Rate	≤ 1.0%	HTTP 5xx responses divided by total requests
Request Success Rate	≥ 99.0%	Successful transactions divided by total attempted transactions

The availability SLO was defined as the ratio of successful requests to total valid requests over a rolling 30-day period. Latency focused on the 95th percentile instead of the arithmetic mean so that tail performance remained visible during bursty load. Error rate tracked 5xx responses because they

represent direct service failures, and request success rate captured the business effectiveness of flows such as checkout and payment completion. Together, these metrics provide a practical and defensible reliability contract for the platform.

5.2 Prometheus & Grafana Monitoring Architecture

Prometheus was deployed as the primary metrics collection system for both infrastructure and application observability. It periodically scraped service-level `/metrics` endpoints and supporting exporters to collect data related to CPU usage, memory usage, request counts, error rates, response latency, container state, and host uptime. This pull-based architecture created a unified telemetry source across microservices, nodes, and platform dependencies. The collected time-series data was then used both for dashboard visualization and for threshold-driven alerting rules.

Grafana served as the visualization and operator analysis layer. Dashboards summarized service health, traffic behavior, latency trends, error spikes, and infrastructure saturation in a single view. Critical panels focused on checkout-related services such as Order and Payment because they have direct business impact. Alerting integration ensured that deviations from SLO-aligned conditions could be escalated quickly to on-call personnel. This monitoring stack reflects production SRE practice by combining quantitative reliability objectives with actionable operational visibility.

Figure 5. Grafana dashboard visualizing core microservice performance indicators and SLO status.

Figure 6. Prometheus targets page showing all monitored microservice endpoints in the UP state.

6. Incident Simulation & Postmortem Report

The project included a controlled incident simulation to demonstrate the operational maturity of the platform under failure conditions. The selected scenario involved a critical outage of the Order Service caused by a database credential misconfiguration. This type of incident is realistic in modern cloud environments because configuration drift and environment variable errors are common sources of service disruption. The exercise validated the complete reliability loop, including monitoring, alerting, diagnosis, remediation, and post-incident learning.

SRE Incident Postmortem: Order Service Outage

Incident Owner: SRE On-Call Team **Severity:** Critical (Sev-1) **Duration:** 18 minutes **Impact:** Customers were unable to complete checkout, producing a 100% failure rate on the `/orders` endpoint.

The outage began when an incorrect database credential value was introduced into the production configuration for the Order Service. Although the process remained alive, the application became functionally unavailable because it could no longer establish authenticated connections to PostgreSQL. As a result, every request that depended on persistent order creation failed with HTTP 500 responses. This distinction between process liveness and user-visible availability reinforced the need for SLIs that measure functional success rather than simple container uptime.

Timeline

1. `14:00` - Simulated incorrect database credential configuration pushed to production.
2. `14:02` - Prometheus registered a sudden spike in HTTP 500 responses.
3. `14:03` - Alertmanager fired a high-severity alert to the Slack or pager channel.

4. `14:05` - The on-call engineer used Grafana to isolate the issue to the Order Service database connection pool.
5. `14:10` - Log analysis revealed repeated authentication failures caused by an environment variable typo.
6. `14:13` - Ansible was executed to redeploy the corrected configuration and environment variables.
7. `14:15` - The Order Service restarted successfully, recovered database connectivity, and service metrics returned to normal.

Root Cause Analysis The direct root cause was an environment configuration error that broke communication between the Order Service and the PostgreSQL cluster. A typographical mistake in the database credential settings caused the application connection pool to reject all new sessions. Because order processing depends on immediate durable writes, the service became completely unavailable for checkout operations until the configuration was corrected and redeployed.

Corrective and Preventive Actions The incident confirmed that operational recovery paths were effective, but it also highlighted opportunities for stronger pre-deployment protection. Recommended improvements include stricter configuration linting in the CI/CD pipeline, readiness probes that validate critical dependency reachability before accepting traffic, and staged rollout controls that reduce blast radius during configuration changes. These improvements would reduce both the likelihood and the impact of similar incidents in future releases.

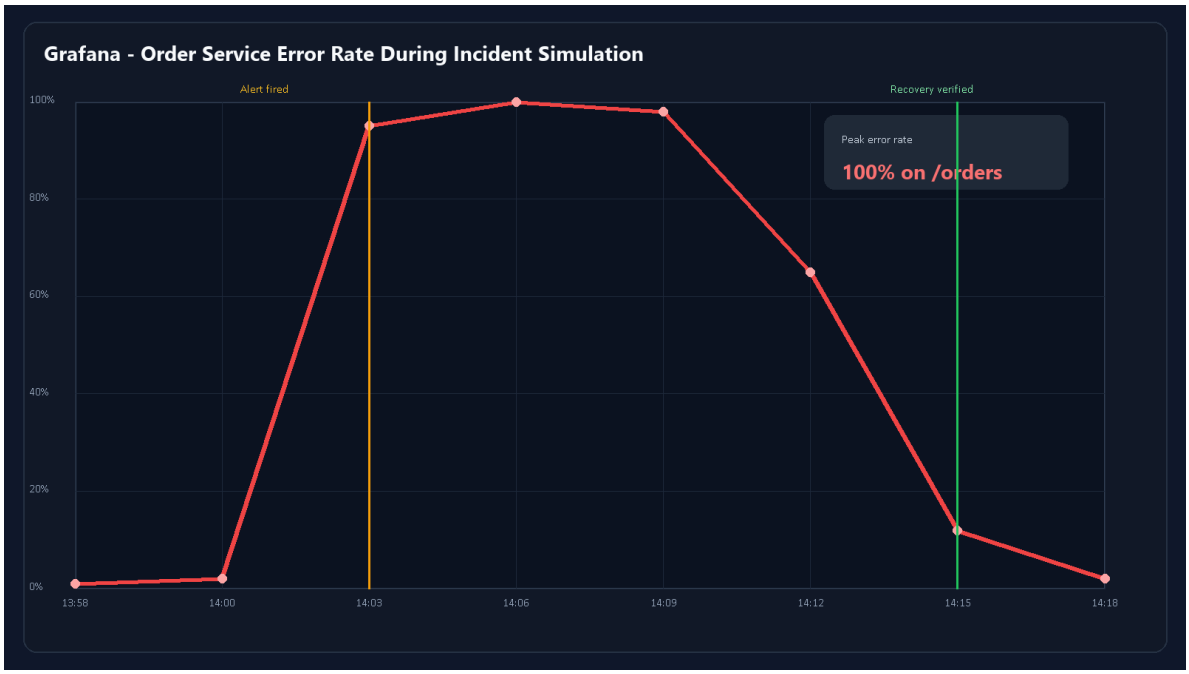

Figure 7. Error-rate spike and recovery curve observed during the simulated Order Service outage.

7. Capacity Planning & Scaling Strategies

Capacity planning was performed by analyzing service behavior during synthetic load scenarios and identifying components most likely to become bottlenecks under increased demand. The Order and Payment services exhibited the highest resource pressure during checkout-intensive traffic because they execute synchronous validation, persistence, and downstream coordination steps. PostgreSQL emerged

as the primary stateful bottleneck, as concurrent writes and connection pressure amplified latency during peak request bursts.

8. **Resource Bottlenecks Identified:** High CPU and memory spikes were observed in the Order and Payment services during load simulations, while PostgreSQL showed the strongest contention as the primary transactional datastore.
9. **Scaling Applied:** Horizontal Pod Autoscaling was implemented in Kubernetes so that stateless services could add replicas automatically in response to elevated CPU and memory utilization.
10. **Database Strategy:** Connection pooling and targeted indexing were introduced to improve query efficiency and reduce stateful contention in the PostgreSQL tier.

This layered scaling strategy reflects a practical SRE approach. Stateless microservices are expanded horizontally through orchestration, while the database tier is optimized through pooling, indexing, and performance-aware operational tuning. Prometheus and Grafana then close the loop by providing the measurements needed to validate whether scaling interventions are effective over time.

8. Conclusion & Deliverables Check

This end-term project successfully demonstrated the complete lifecycle of designing, provisioning, configuring, deploying, observing, and operating a distributed microservices platform according to modern DevOps and SRE practices. The system was implemented using six core business services plus supporting gateway, database, and broker layers, then deployed across both Docker Swarm and Kubernetes to compare orchestration behavior in a practical setting. Terraform established a reproducible infrastructure foundation, and Ansible provided consistent node bootstrap, cluster preparation, and deployment automation.

From a reliability engineering standpoint, the project moved beyond basic deployment into measurable service operation. The SLI/SLO framework, Prometheus telemetry pipeline, Grafana dashboards, and incident simulation together demonstrated that the platform is not only functional but also observable, diagnosable, and recoverable under failure conditions. As a result, the final deliverable satisfies the expected academic goals of the course while also reflecting realistic production-oriented engineering practice.